

Hybrid-Grid: Hệ thống biên dịch phân tán với lập lịch tác vụ dựa trên Contextual Bandit trên cụm máy không đồng nhất

Lê Đức Hiếu^a, Nguyễn Trung Kiên^a, Nguyễn Trọng Khánh^{a,*}

^aHọc viện Công nghệ Bưu chính Viễn thông, Hà Nội, Việt Nam

Abstract

Bài báo này trình bày Hybrid-Grid, một hệ thống biên dịch phân tán mã nguồn mở cho C/C++ trên cụm máy trạm không đồng nhất, với trọng tâm nghiên cứu là bài toán lập lịch tác vụ trực tuyến. Các hệ thống build phân tán hiện có đều dựa vào heuristic tĩnh — round-robin, least-loaded, hoặc chấm điểm năng lực — và không học từ kết quả thực thi, trong khi đo lường của chúng tôi cho thấy thời gian biên dịch trên cụm không đồng nhất có phương sai rất lớn (tỉ số P99/P50 xấp xỉ 29×). Để khai thác tín hiệu này, chúng tôi mô hình hoá quyết định lập lịch như một bài toán contextual bandit và triển khai, so sánh sáu bộ lập lịch trong cùng một hệ thống thực: ba heuristic (LeastLoaded, Power-of-Two-Choices, HEFT) và ba bộ học trực tuyến (ϵ -greedy, LinUCB, và Hybrid-LinUCB — biến thể do chúng tôi đề xuất, kết hợp warm-start theo heuristic và thành phần phạt tải thời gian thực vào điểm UCB). Thí nghiệm trên workload biên dịch CPython (~293 tác vụ/build) chạy trên cụm Docker 5 worker không đồng nhất, theo thiết kế khối ngẫu nhiên hoá đầy đủ với 10 lần lặp và kiểm định ghép cặp (Wilcoxon signed-rank, hiệu chỉnh Holm–Bonferroni, effect size Cliff’s delta), cho kết quả hai mặt: Hybrid-LinUCB vượt có ý nghĩa thống kê các bộ học yếu hơn (nhanh hơn P2C 5,26 s và LinUCB thuần 6,80 s; $p = 0,003$; $|d| = 1,0$) nhưng chỉ ngang bằng heuristic LeastLoaded ($p = 0,65$) — một kết quả âm trung thực được củng cố bằng ablation warm-bandit chứng minh thể hoà là bản chất của workload dừng chứ không phải do khởi động lạnh. Đóng góp phương pháp luận quan trọng không kém: quy trình đo nghiêm ngặt đã phát hiện và loại bỏ một kết quả dương tính giả (ưu thế biểu kiến 7,9%; $p = 0,0001$) mà thiết kế thí nghiệm đơn giản hơn đã sinh ra do nhiễu thứ tự chạy. Các kết quả này xác định rõ điều kiện để lập lịch học máy có giá trị thực trong hệ thống build phân tán, và chỉ ra cache-affinity cùng môi trường không dừng là hướng khai thác triển vọng nhất.

Keywords: biên dịch phân tán, lập lịch tác vụ, contextual bandit, LinUCB, máy không đồng nhất, học tăng cường trực tuyến

1. Giới thiệu

Biên dịch phân tán là một thực tế thường nhật của kỹ nghệ phần mềm hiện đại. Các dự án C/C++ lớn như LLVM, Chromium hay nhân Linux chứa hàng chục nghìn đơn vị dịch (translation unit); một lần build sạch trên máy đơn có thể kéo dài hàng giờ. Các hệ thống như distcc [15], Bazel Remote Build Execution và Incredibuild giải quyết vấn đề bằng cách đẩy tác vụ biên dịch lên một bể worker từ xa. Bể worker này về bản chất là *không đồng nhất*: laptop của lập trình viên, máy chủ CI Linux, node ARM trên cloud, máy build chuyên dụng — khác nhau về số lõi CPU, bộ nhớ, kiến trúc tập lệnh và tải tức thời.

Về lý thuyết, gán tác vụ lên máy không đồng nhất để cực tiểu hoá makespan là bài toán $R||C_{\max}$ (unrelated parallel machines): NP-khó, không thể xấp xỉ tốt hơn tỉ lệ $3/2$ trừ khi $P = NP$, với cận trên xấp xỉ tốt nhất hiện biết là 2 [1]. Trong thiết lập trực tuyến — tác vụ đến tuần tự và phải được gán ngay — tình hình còn khó hơn: list scheduling của Graham [2] đạt tỉ lệ cạnh tranh $2 - 1/m$ trên máy đồng nhất, nhưng với máy không đồng nhất không tồn tại cận cạnh tranh phổ quát chặt. Vì vậy, các hệ thống sản xuất dựa vào heuristic: round-robin, least-loaded, hoặc Power-of-Two-Choices (P2C) [3]. Đặc điểm chung của các heuristic

này là chúng ra quyết định từ độ dài hàng đợi hoặc điểm năng lực tĩnh và *không bao giờ học* từ kết quả quan sát được.

Đo lường của chúng tôi trên một build CPython (~293 tác vụ biên dịch/build) trên cụm 5 worker không đồng nhất cho thấy hai hiện tượng đáng chú ý: (i) tỉ số giữa phân vị P99 và P50 của thời gian biên dịch lên tới **29×**, và (ii) một worker “tốt nhất” hấp thụ **33% tổng số dispatch** dưới cả hai heuristic tĩnh. Phương sai lớn kết hợp với phân phối tập trung gợi ý rằng một bộ lập lịch *quan sát kết quả biên dịch và điều chỉnh quyết định tương lai* — một bộ học trực tuyến — có thể cải thiện makespan và độ trễ đuôi.

Tuy nhiên, các tiếp cận học tăng cường (RL) cho lập lịch đã công bố (Decima [12], DeepRM [11]) đều đòi hỏi huấn luyện trước trong trình mô phỏng — thứ không tồn tại cho hệ thống build với trình biên dịch thật và phần cứng nhiễu. Lớp thuật toán *contextual multi-armed bandit* [7, 8] là lựa chọn thay thế ít được khai phá hơn: mỗi quyết định lập lịch là một bài toán một bước, học trực tuyến từ phần thưởng quan sát được, với cận regret lý thuyết dạng $O(\sqrt{T})$ dưới giả thiết tuyến tính [10]. Thuật toán tiêu biểu nhất của họ này là LinUCB [9].

Trong bài báo này, chúng tôi xây dựng trọn vẹn một hệ thống biên dịch phân tán (Hybrid-Grid) và dùng nó làm nền tảng thí nghiệm để trả lời câu hỏi: *một contextual bandit có vượt được heuristic tĩnh trong lập lịch biên dịch phân tán thực tế hay*

*Tác giả liên hệ (giảng viên hướng dẫn)

Email address: [email liên lạc] (Nguyễn Trọng Khánh)

không? Đóng góp chính của nghiên cứu gồm:

- **C1.** Một hệ thống biên dịch phân tán mã nguồn mở hoàn chỉnh viết bằng Go: CLI `hgbuild` thay thế trực tiếp `gcc/clang`, coordinator `hg-coord` với registry, circuit breaker và dashboard thời gian thực, worker `hg-worker` hỗ trợ thực thi native và Docker cross-compile; giao tiếp gRPC/HTTP2 với TLS/mTLS, tự khám phá worker qua mDNS, cache nội dung định địa chỉ bằng xxhash, và khả năng quan sát đầy đủ (Prometheus, OpenTelemetry).
- **C2.** Sáu bộ lập lịch cùng cài đặt sau một giao diện `LearningScheduler` thống nhất hỗ trợ phản hồi phần thưởng trực tuyến, trong đó có **Hybrid-LinUCB** — biến thể lai do chúng tôi đề xuất, bổ sung của sổ warm-start học thụ động và thành phần phạt tải thời gian thực $\lambda \cdot \text{LoadRatio}$ vào điểm UCB để khắc phục hai chế độ thất bại (cold start và phản hồi trễ) quan sát được của bandit thuần.
- **C3.** Một đường ống đo lường phát sinh một bản ghi JSON Lines 27 trường cho mỗi tác vụ được dispatch — bao phủ ngữ cảnh worker tại thời điểm quyết định, phân rã độ trễ, và introspection của bộ học — cho phép phân tích offline tái lập được.
- **C4.** Một đánh giá thực nghiệm có kiểm soát nhiều (thiết kế khối ngẫu nhiên hoá, 10 lần lặp, kiểm định ghép cặp với hiệu chỉnh so sánh bội) kèm ablation warm-bandit, cho một kết quả âm trung thực — bandit ngang heuristic mạnh nhất trên workload dùng — và một bài học phương pháp luận: thiết kế nghiêm ngặt đã bắt được kết quả dương tính giả mà thiết kế đơn giản sinh ra.

Phần còn lại của bài báo được tổ chức như sau. Mục 2 điểm lại các công trình liên quan. Mục 3 phát biểu bài toán lập lịch trực tuyến dưới khung contextual bandit. Mục 4 mô tả kiến trúc hệ thống và các thuật toán lập lịch, đặc biệt là thiết kế Hybrid-LinUCB. Mục 5 trình bày thiết lập thí nghiệm và quy trình thống kê. Mục 6 báo cáo kết quả định lượng. Mục 7 thảo luận phát hiện chính, ý nghĩa thực tiễn, hạn chế và các mối đe dọa đến tính hợp lệ. Mục 8 kết luận.

2. Công trình liên quan

2.1. Lập lịch heuristic trên máy không đồng nhất

Bài toán $R||C_{\max}$ được Lenstra, Shmoys và Tardos [1] chứng minh NP-khó với cận dưới xấp xỉ $3/2$ và thuật toán xấp xỉ tỉ lệ 2; khoảng cách này vẫn mở sau hơn 35 năm. Trong thực hành, Power-of-Two-Choices [3] là heuristic nổi bật: lấy mẫu ngẫu nhiên hai server và chọn server ít tải hơn giảm tải cực đại từ $\Theta(\log n / \log \log n)$ xuống $\Theta(\log \log n)$ trên cụm đồng nhất. Sparrow [4] mở rộng P2C với late binding cho lập lịch phi tập trung độ trễ thấp. HEFT [5] là thuật toán kinh điển cho DAG tác vụ trên máy không đồng nhất, xếp hạng tác vụ theo upward rank và gán theo thời gian hoàn thành sớm nhất; chúng tôi điều chỉnh HEFT cho luồng tác vụ trực tuyến bằng ước lượng EWMA thời gian biên dịch. Điểm chung của cả nhóm: quyết định dựa trên trạng thái hàng đợi hoặc năng lực tĩnh, không quan sát kết quả từng tác vụ.

2.2. Multi-armed bandit và contextual bandit

Khung bandit cổ điển với ϵ -greedy và cập nhật trung bình mẫu được trình bày hệ thống trong Sutton và Barto [6]; UCB1 của Auer và cộng sự [16] đưa nguyên lý “lạc quan trước bất định” vào chọn cánh tay. Slivkins [7] và Lattimore–Szepesvári [8] hệ thống hoá lý thuyết regret. LinUCB [9] mở rộng sang thiết lập có ngữ cảnh: mỗi cánh tay duy trì mô hình tuyến tính (A_a, b_a) , chọn cánh tay theo điểm $\hat{\theta}_a^\top x + \alpha \sqrt{x^\top A_a^{-1} x}$. Cận regret $\tilde{O}(\sqrt{Td})$ được chứng minh cho biến thể SupLinUCB [10]; đáng chú ý là bound này *không* áp dụng trực tiếp cho LinUCB Algorithm 1 mà các hệ thống thực (kể cả chúng tôi) triển khai — một điểm chúng tôi trình bày trung thực ở Mục 7.

2.3. Học máy cho lập lịch hệ thống

Decima [12] dùng mạng nơ-ron đồ thị với REINFORCE để lập lịch job Spark, đạt cải thiện tới $1.5\times$ thời gian hoàn thành job, nhưng đòi hỏi hàng nghìn episode huấn luyện trong trình mô phỏng trước khi triển khai. DeepRM [11] là công trình sớm nhất áp policy-gradient cho quản lý tài nguyên, cũng phụ thuộc mô phỏng. Quasar [13] dùng collaborative filtering (học có giám sát, không có exploration) để dự đoán hiệu năng job trên từng loại máy. Resource Central [14] dùng random forest dự đoán vòng đời VM trong Azure — học có giám sát offline ở quy mô sản xuất. Khoảng trống mà nghiên cứu này nhắm đến: *học trực tuyến, không cần mô phỏng, áp dụng cụ thể cho biên dịch phân tán* — theo hiểu biết của chúng tôi, chưa có công bố nào áp contextual bandit cho bài toán này và mô tả các cạm bẫy triển khai phát sinh khi tương tác với phân phối độ trễ biên dịch thật.

2.4. Hệ thống build phân tán

Các hệ thống build sản xuất chia ba lớp: hệ *cache-first* (ccache, sccache, Bazel RBE) ưu tiên tra cứu cache theo hash, chọn worker bằng round-robin hoặc least-loaded; hệ *chăm điểm năng lực* (distcc [15]) xếp hạng worker theo năng lực tĩnh; hệ *nhận thức DAG* (Bazel, Buck2) dùng đồ thị build để trì hoãn quyết định nhưng trong mỗi lớp DAG vẫn lập lịch theo năng lực. Không hệ nào học trực tuyến. Các công trình gần nhất dùng ML offline dự đoán thời gian tác vụ rồi dispatch bằng heuristic trên dự đoán đó — tức lập lịch bằng học có giám sát, không phải bandit.

3. Phát biểu bài toán

3.1. Bối cảnh hệ thống

Hybrid-Grid chia một quá trình biên dịch C/C++ thành hàng trăm tác vụ độc lập (mỗi file `.c/.cpp` sau tiền xử lý là một tác vụ) và phân phối tới cụm worker không đồng nhất. Coordinator giữ registry các worker đang sống — mỗi worker được mô tả bởi đặc trưng tĩnh (số lõi CPU, bộ nhớ, kiến trúc native, hệ điều hành) và bộ đếm động (số tác vụ đang chạy, độ trễ RPC gần đây) — và quyết định gửi tác vụ nào tới worker nào. Đây là điểm ảnh hưởng lớn nhất tới makespan và là trọng tâm của nghiên cứu.

3.2. Mô hình hoá contextual bandit

Tại thời điểm quyết định t , coordinator quan sát véc-tơ ngữ cảnh $x_{t,a} \in \mathbb{R}^d$ cho mỗi worker hợp lệ a (đã lọc worker không khoẻ, circuit breaker mở, hoặc chậm trễ song song), chọn một hành động a_t , dispatch tác vụ, và khi tác vụ hoàn tất nhận được phần thưởng vô hướng

$$r_t = -\log(1 + t_{\text{compile}}^{(t)}), \quad (1)$$

trong đó $t_{\text{compile}}^{(t)}$ là thời gian biên dịch do worker báo cáo (mili-giây). Phép biến đổi log là lựa chọn kỹ nghệ nhằm nén đuôi nặng của phân phối (P99/P50 $\approx 29\times$): không nén, một outlier 24 giây sẽ áp đảo các cập nhật trung bình mẫu. Tác vụ thất bại nhận $r = -\log(1 + T_{\text{timeout}})$ — trường hợp xấu nhất hệ thống có thể quan sát — để bộ học tự động hạ giá worker hỏng dai dẳng. Chúng tôi lưu ý trung thực rằng dạng phần thưởng này không có nguồn peer-review trực tiếp; dạng có biện luận lý thuyết gần nhất là phần thưởng tích phân thời gian của Decima [12].

Chúng tôi chọn khung bandit thay vì MDP đầy đủ vì hai lý do. Thứ nhất, các tác vụ biên dịch gần như độc lập — khi đặc trưng độ sâu hàng đợi đã nằm trong véc-tơ ngữ cảnh, phần ghép nối trạng thái còn lại giữa các quyết định liên tiếp là nhỏ [7]. Thứ hai, MDP đòi hỏi quy hồi makespan về từng quyết định dispatch — bài toán credit assignment với phần thưởng thưa qua hàng trăm bước, chính là chế độ mà các phương pháp policy-gradient cần trình mô phỏng [11, 12] mà chúng tôi chủ đích không có.

4. Hệ thống và phương pháp

4.1. Kiến trúc Hybrid-Grid

Hệ thống gồm ba thành phần chính giao tiếp qua gRPC trên HTTP/2 (TLS/mTLS tùy chọn):

- **hgbuild (CLI)**: thay thế trực tiếp gcc/clang và bọc được make/ninja. Nó tiền xử lý mã nguồn cục bộ, băm đầu ra tiền xử lý bằng xxhash để tra cứu cache, gửi tác vụ lên coordinator, và tự động fallback biên dịch cục bộ khi coordinator không khả dụng.
- **hg-coord (Coordinator)**: bộ điều phối trung tâm gồm registry worker (đăng ký qua handshake gRPC, heartbeat 10 giây), bộ lập lịch (Mục 4.2), circuit breaker theo từng worker để cô lập lỗi, số liệu Prometheus, tracing OpenTelemetry, và dashboard web thời gian thực qua WebSocket.
- **hg-worker (Worker)**: thực thi biên dịch ở hai chế độ — native (gọi trực tiếp gcc/clang) hoặc Docker (cross-compile qua ảnh dockcross); duy trì cache nội dung định địa chỉ cục bộ (tăng tốc $\sim 10\times$ khi trúng cache); tự công bố qua mDNS để coordinator khám phá không cần cấu hình.

Ngoài đường C/C++ là trọng tâm, hệ thống còn hỗ trợ build Flutter Android phân tán và dịch cờ GCC/Clang sang MSVC; các đường này không đi qua bộ lập lịch học nên nằm ngoài phạm vi đánh giá.

Table 1: Véc-tơ đặc trưng 12 chiều của (Hybrid-)LinUCB.

#	Đặc trưng	Chuẩn hoá
0	bias	hằng số 1,0
1	log kích thước nguồn tiền xử lý	$\frac{\log(1+s)}{\log(1+4 \text{ MiB})}$, chặn 1
2	kiến trúc đích = x86_64	one-hot 0/1
3	kiến trúc đích = arm64	one-hot 0/1
4	số lõi CPU worker	cores/16, chặn 1
5	bộ nhớ worker	mem/64 GiB, chặn 1
6	kiến trúc native khớp đích	0/1
7	áp lực hàng đợi	$\frac{\text{active}}{\text{max_parallel}}$, chặn 1
8	độ trễ RPC gần đây	ms/100, chặn 1
9	tác vụ là C++	0/1
10	log kích thước nguồn thô	$\frac{\log(1+s_{\text{raw}})}{\log(1+1 \text{ MiB})}$, chặn 1
11	tỉ lệ thành công (Laplace)	$(s+1)/(s+f+2)$

4.2. Khung lập lịch hợp nhất

Giao diện Scheduler cung cấp `Select(buildType, arch, clientOS) → (Worker, error)`. Giao diện mở rộng `LearningScheduler` thêm `SelectWithDispatchInfo` (trả kèm Q-value và cờ exploration để ghi log) và `RecordOutcome`. `Handler Compile()` của coordinator dùng type assertion để chỉ phản hồi phần thưởng khi bộ học được cấu hình — các heuristic không phải sửa đổi. Sáu bộ lập lịch được chọn qua cờ CLI `-scheduler`: (1) **LeastLoaded**; (2) **P2C** [3]; (3) **HEFT** với ước lượng EWMA ($\alpha = 0,3$) [5]; (4) **ε -greedy** với trung bình mẫu theo worker và $\varepsilon = 0,1$ [6] — một chính sách mù đặc trưng; (5) **LinUCB** (Mục 4.3); (6) **Hybrid-LinUCB** (Mục 4.4). Cả ba bộ học đều có fast-path đơn ứng viên: khi chỉ còn một worker hợp lệ, trả về ngay không cần đại số ma trận — khắc phục mức phạt 41% quan sát được của pipeline lọc P2C trên cụm 1 worker.

4.3. Bộ lập lịch LinUCB

Theo Li và cộng sự [9], với mỗi worker a chúng tôi duy trì $A_a \in \mathbb{R}^{d \times d}$ khởi tạo I_d và $b_a \in \mathbb{R}^d$ khởi tạo $\mathbf{0}$. Ước lượng ridge-regression hiện hành là $\hat{\theta}_a = A_a^{-1} b_a$. Tại vòng t , điểm UCB của mỗi worker hợp lệ là

$$p_{t,a} = \hat{\theta}_a^\top x_{t,a} + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}}, \quad (2)$$

chọn $a_t = \arg \max_a p_{t,a}$; khi có phần thưởng, cập nhật $A_{a_t} \leftarrow A_{a_t} + x x^\top$, $b_{a_t} \leftarrow b_{a_t} + r x$.

Véc-tơ đặc trưng ($d = 12$). Bảng 1 liệt kê bố cục đặc trưng; mọi đặc trưng được đưa xấp xỉ về $[0, 1]$ để giữ $\|x\|$ bị chặn theo quy ước tuyến tính của [10]. Hai quyết định thiết kế đáng lưu ý. Thứ nhất, thiết kế ban đầu dùng one-hot ba chiều cho loại build (CPP/Flutter/Unity); vì đường `Compile()` hiện chỉ phát sinh CPP, các chiều này suy biến (cột CPP trùng tuyến tính với bias, hai cột còn lại vĩnh viễn bằng 0) và bị loại bỏ. Thứ hai, đặc trưng tỉ lệ thành công nếu dùng dạng thô với prior lạc quan sẽ bằng đúng 1,0 cho mọi worker trên workload thành công hoàn toàn — lại trùng tuyến tính với bias — nên chúng tôi áp Laplace smoothing để cột luôn phụ thuộc kinh nghiệm.

Cập nhật nghịch đảo Sherman–Morrison. Mỗi cập nhật là hàng-1: $A_{\text{new}} = A_{\text{old}} + xx^T$. Nghịch đảo lại từ đầu tốn $O(d^3)$; công thức Sherman–Morrison [17] giảm còn $O(d^2)$:

$$A_{\text{new}}^{-1} = A_{\text{old}}^{-1} - \frac{A_{\text{old}}^{-1}xx^TA_{\text{old}}^{-1}}{1 + x^TA_{\text{old}}^{-1}x}. \quad (3)$$

Một unit test đối chiếu nghịch đảo được cache với phép nghịch đảo tươi sau 50 cập nhật ngẫu nhiên; sai lệch từng phần tử dưới 10^{-6} .

Ba cam bẫy triển khai. Lần chạy đầu tiên của LinUCB ($\alpha = 1,0$) cho kết quả *tệ hơn mọi baseline* (158 s trên cụm 5 worker, so với P2C 94 s). Rà soát mã độc lập tìm ra ba lỗi: (i) *rò rỉ đích (target leakage)* — véc-tơ đặc trưng được dựng lại tại thời điểm cập nhật, dùng trạng thái worker *sau khi* tác vụ hoàn thành thay vì tại thời điểm dispatch; (ii) *đa cộng tuyến* của one-hot loại build như mô tả ở trên; (iii) *lệch thang phần thưởng* — biên độ phần thưởng ($|r| \approx 7$) áp đảo bonus khám phá, khiến tỉ lệ exploration thực tế tụt còn 1,7%. Sau khi sửa (cache véc-tơ x tại Select theo TaskID, loại chiều suy biến, hạ α về 0,5), makespan hồi phục về 94 s — **toàn bộ 64 giây phục hồi đến từ kỷ luật triển khai, không từ thay đổi thuật toán.**

4.4. Hybrid-LinUCB: thiết kế đề xuất

Quan sát hành vi của LinUCB thuần cho thấy hai chế độ thất bại còn lại: *cold start* — trong các dispatch đầu tiên, ước lượng $\hat{\theta}_a$ chưa có thông tin nên các quyết định gần như ngẫu nhiên; và *phản hồi trễ* — phần thưởng về sau vài giây, trong cửa sổ đó bandit không biết một worker đang bị dôn việc và tiếp tục dôn thêm. Hybrid-LinUCB khắc phục cả hai bằng hai cơ chế trực giao, cấu hình qua `-warm-start` (mặc định $N = 100$) và `-load-penalty` (mặc định $\lambda = 0,5$):

(a) **Warm-start học thụ động.** Trong N dispatch đầu, quyết định được giao cho heuristic least-loaded, nhưng bộ học *vẫn* tính và lưu véc-tơ đặc trưng của worker được chọn; khi phần thưởng về, cập nhật Sherman–Morrison diễn ra bình thường. Bandit “quan sát và học” trong khi mất quyền quyết định — sau N dispatch nó tiếp quản với ma trận đã được làm ấm bằng các quan sát thật.

(b) **Điểm lai với phạt tải.** Sau warm-start, điểm chọn worker là

$$Q_{\text{hybrid}} = \underbrace{\hat{\theta}_a^T x}_{\text{giá trị đã học}} + \underbrace{\alpha \sqrt{x^T A_a^{-1} x}}_{\text{bonus khám phá}} - \underbrace{\lambda \cdot \text{LoadRatio}_a}_{\text{phạt quá tải}}. \quad (4)$$

LoadRatio vốn đã là đặc trưng $x[7]$, nên về lý thuyết LinUCB có thể tự học trọng số âm cho nó; thành phần phạt đóng vai trò *prior thủ công* dùng thông tin thời gian thực (đếm tác vụ theo mili-giây, như LeastLoaded) để chặn hành vi dôn việc *trước khi* mô hình tích đủ dữ liệu tự nhận ra — đây là luận điểm “lại” trung tâm của thiết kế. Ràng buộc $\lambda \in [0, 5]$ vì λ quá lớn khiến phạt áp đảo mọi ước lượng đã học, thoái hoá về LeastLoaded. Khi cả hai tham số bằng 0, hành vi trùng từng bit với LinUCB thuần — bảo đảm so sánh benchmark là cùng code path, chỉ khác cấu hình.

Table 2: Makespan (s), chạy đơn — chỉ mang tính bối cảnh, không dùng để kết luận so sánh.

Cấu hình	LL	P2C	ε -g	LinUCB ^{lỗi}	LinUCB ^{sửa}	HEFT
1w-4.0cpu	92	130	146	129	131	129
3w-hetero	123	85	142	103	108	135
5w-hetero	152	94	119	158	94	144

4.5. Đường ống đo lường

Mỗi lời gọi `Compile()` hoàn tất phát sinh một bản ghi JSON Lines 27 trường: định danh (task, loại build, scheduler), ngữ cảnh worker tại dispatch (lỗi, RAM, kiến trúc, số tác vụ đang chạy, nguồn khám phá), tác vụ (kích thước nguồn thô/tiền xử lý, kiến trúc đích), phân rã độ trễ (queue/compile/RPC/tổng), kết cục (thành công, exit code, trúng cache), và introspection bộ học (Q-value tại dispatch, cờ exploration). File nạp thẳng vào pandas bằng `read_json(lines=True)`; lược đồ được kiểm chứng 100% trên 1.746 bản ghi giai đoạn đo sơ bộ.

5. Thí nghiệm

5.1. Thiết lập

Workload: build sạch CPython qua make với hgbuild làm compiler driver — ~293 đơn vị dịch/build (tải nhẹ) và biến thể ~371 đơn vị dịch (tải nặng). **Cụm máy:** Docker Compose trên một host, tổng CPU cố định 4,0 lõi chia không đều cho 5 worker (0,5/0,6/0,8/1,0/1,1 cpu) qua giới hạn cgroup; các cấu hình 1 worker và 3 worker dùng cho đo sơ bộ. **Chỉ số:** makespan (wall-clock), phân phối dispatch theo worker, phân vị P50/P95/P99 thời gian biên dịch.

5.2. Thiết kế khối ngẫu nhiên hoá và phân tích thống kê

Đo lường sơ bộ cho thấy nền tảng Docker trên một host có sẵn nhiều chạy-lại đáng kể; các phép so sánh single-run vì vậy không đủ để kết luận. Quan trọng hơn, bố trí “scheduler-major” (chạy hết các lần lặp của một scheduler rồi mới sang scheduler khác) *trộn lẫn* danh tính scheduler với trôi dạt chậm của host (nhiệt, tải nền). Chúng tôi thiết kế lại theo **randomized complete block design**: 10 round, mỗi round chạy cả bốn scheduler đúng một lần theo thứ tự xáo trộn có seed riêng từng round; kèm một build warm-up bị loại, rào chắn chờ đủ 5/5 worker đăng ký, cooldown cố định giữa các build, đo thời gian dưới-giây, và xoá cache biên dịch trước mỗi build. Mỗi round là một khối thống kê, cho phép kiểm định **ghép cặp**: Friedman omnibus [20], Wilcoxon signed-rank một phía [18], hiệu chỉnh Holm–Bonferroni [19], effect size Cliff’s delta [21], và khoảng tin cậy bootstrap.

6. Kết quả

6.1. Đo sơ bộ đơn lần: heuristic mạnh, bộ học ngây thơ yếu

Ba quan sát từ Bảng 2: (i) P2C cải thiện 1,45–1,62× so với LeastLoaded trên cụm không đồng nhất, khớp dự đoán lý thuyết [3]; (ii) ε -greedy mù đặc trưng thua *mọi* heuristic — bộ học trung thành với “worker có trung bình phần thưởng tốt nhất”

Table 3: Makespan (s), 10 khối, cụm 5 worker không đồng nhất, tải nhẹ (~293 tác vụ/build). Wilcoxon ghép cặp + Holm so với Hybrid-LinUCB.

Scheduler	Median	Mean	Std	vs. hybrid
LeastLoaded	40,24	40,38	1,02	hoà ($p = 0,65$)
Hybrid-LinUCB	40,52	40,56	1,06	—
P2C	45,60	46,01	1,39	thắng ($p = 0,003$)
LinUCB	47,48	47,11	1,45	thắng ($p = 0,003$)

Table 4: Makespan (s) trên tải nặng (~371 tác vụ/build).

Scheduler	Median	Mean	Std	vs. hybrid
LeastLoaded	48,65	51,36	6,78	hoà ($p = 0,82$)
Hybrid-LinUCB	50,64	55,42	16,60	—
P2C	53,48	55,22	5,08	$p_{\text{holm}} = 0,16$
LinUCB	60,20	59,45	6,35	$p_{\text{holm}} = 0,16$

rời đồn việc vào đó, làm lệch tải nặng hơn cả LeastLoaded (tỉ số dispatch top:bottom 13,2:1 so với P2C 8,6:1); (iii) LinUCB có lỗi là scheduler tệ nhất trên 5 worker (158 s), phục hồi về 94 s sau ba bản vá triển khai (Mục 4.3) — với tỉ lệ exploration tăng từ 1,7% lên 25,1% và lệch tải giảm từ 97:1 về 13,9:1.

6.2. Kết quả chính: so sánh có kiểm soát nhiều

Friedman omnibus cho $\chi^2 = 24,60$, $p = 2 \times 10^{-5}$: các scheduler khác nhau có ý nghĩa (Bảng 3). Hybrid-LinUCB vượt P2C ($\Delta = -5,26$ s) và LinUCB thuần ($\Delta = -6,80$ s) với effect size cực đại ($|d| = 1,0$ — thắng ở cả 10/10 khối), nhưng **thống kê ngang bằng LeastLoaded** ($\Delta = +0,09$ s; $d = +0,10$). Về độ trễ đuôi P99, Friedman cho $p = 0,169$ — không phát hiện khác biệt nào giữa bốn scheduler; ưu thế đuôi 2,3% từng thấy ở lần chạy đơn không lặp lại được.

Một kết quả dương tính giả bị bắt. Phân tích đầu tiên của cùng bộ dữ liệu, đo theo bố trí scheduler-major, cho thấy Hybrid-LinUCB vượt LeastLoaded 7,9% với $p = 0,0001$. Khi loại confound thứ tự chạy bằng thiết kế khối, ưu thế đó biến mất hoàn toàn ($p = 0,65$): chênh lệch biểu kiến là *artifact* của trôi nhiệt/tải nền — LeastLoaded đã bị đo khi host ở trạng thái khác. Đây là minh chứng trực tiếp rằng thiết kế thí nghiệm nghiêm ngặt tự bắt được kết quả dương tính giả do thiết kế cấu thả sinh ra.

Ở tải nặng (Bảng 4), LeastLoaded nhanh nhất theo median và ưu thế của hybrid trước P2C/LinUCB không sống sót qua hiệu chỉnh Holm. Độ lệch chuẩn lớn của hybrid (16,6) đến từ đúng một outlier (một round đạt 101,93 s, gấp đôi bình thường): chuỗi quyết định khám phá tệ lúc khởi động lạnh bộc lộ **rủi ro đuôi** của bandit mà heuristic tĩnh không có.

6.3. Ablation warm-bandit: thể hoà là bản chất

Giả thuyết tự nhiên giải thích thể hoà: benchmark khởi động lại coordinator mỗi build nên bandit luôn khởi động lạnh. Chúng tôi kiểm chứng bằng cách giữ **một** coordinator sống qua $K = 8$ build tuần tự (trạng thái bandit là singleton trong bộ nhớ, tích lũy xuyên suốt), lặp 6 session độc lập. Kết quả bác bỏ giả thuyết: xu hướng makespan theo chỉ số build **phẳng** (Spearman $\rho = -0,079$; $p = 0,59$); build thứ 8 không nhanh hơn build thứ

nhất ($\Delta = +0,21$ s; $p = 0,42$); và bandit đã ấm vẫn hoà LeastLoaded ($\Delta = -0,19$ s; $p = 0,50$). Lý giải: mỗi build có ~293 tác vụ trong khi cửa sổ warm-start chỉ 100 — bandit hội tụ *ngay trong một build*; với không gian đặc trưng 12 chiều và workload dừng, ma trận A_a bão hoà từ build đầu, các build sau không còn thông tin mới. Thể hoà do đó không phải hệ quả của khởi động lạnh mà là bản chất: trên workload biên dịch dừng với worker tĩnh, LeastLoaded đã gần tối ưu và không có cấu trúc ẩn nào cho bandit khai thác vượt lên.

6.4. Độ nhạy siêu tham số

Quét $\alpha \in \{0,1; 0,5; 1,0; 2,0\}$ trên cấu hình 5 worker (sau sửa lỗi) cho makespan 98/94/–/96 giây — mọi giá trị trong khoảng đều cạnh tranh với P2C, chênh lệch nằm trong biên nhiễu single-run. Bài học cho người thực hành: kỷ luật sửa lỗi triển khai quan trọng hơn nhiều so với tinh chỉnh α . Chi phí tính toán của bandit không đáng kể: bước chấm điểm 12 chiều mất vài micro-giây mỗi worker, bị áp đảo bởi RTT gRPC của chính thao tác dispatch.

7. Thảo luận

7.1. Phát hiện chính

Kết quả hai mặt của nghiên cứu có thể tóm trong ba mệnh đề. *Thứ nhất*, heuristic tĩnh mạnh đáng ngạc nhiên: LeastLoaded — không học, không tham số — ngang hoặc vượt mọi bộ học trên workload biên dịch dừng. *Thứ hai*, trong nội bộ các bộ học, ngũ cảnh và cơ chế lai có giá trị rõ rệt: Hybrid-LinUCB thắng tuyệt đối (10/10 khối) trước cả P2C lẫn LinUCB thuần, chúng tỏ warm-start và phạt tải khắc phục đúng hai chế độ thất bại đã nhận diện. *Thứ ba*, phần lớn khoảng cách hiệu năng ban đầu của bandit không nằm ở thuật toán mà ở cạm bẫy triển khai — target leakage, đa cộng tuyến đặc trưng, lệch thang phần thưởng — những lỗi mà tài liệu thuật toán không cảnh báo và chỉ lộ ra khi chạy trên hệ thống thật.

7.2. Ý nghĩa thực tiễn

Với người xây dựng hệ thống build phân tán, kết quả khuyến nghị: dùng LeastLoaded/P2C làm mặc định trên workload build sạch; chỉ đầu tư vào lập lịch học khi môi trường có cấu trúc mà heuristic bỏ qua. Phân tích của chúng tôi chỉ ra hai môi trường như vậy: (i) *cache-affinity* — khi worker đã giữ artifact của file trong cache cục bộ, gửi lại file đó cho đúng worker biến một lần biên dịch thành cache hit gần tức thời; tín hiệu này tương quan với đặc trưng lịch sử dispatch mà LeastLoaded mù hoàn toàn, và chỉ hoạt động trong kịch bản incremental build (không phải clean build mà nghiên cứu này đo); (ii) *môi trường không dừng* — worker suy giảm hiệu năng do thermal throttling hay tải nền, nơi bộ học quan sát kết quả có thể phản ứng còn heuristic năng lực tĩnh thì không.

7.3. Hạn chế

(1) *Một workload*: toàn bộ đánh giá dùng CPython — một codebase C duy nhất; các codebase C++ nặng template (Boost, Eigen, LLVM) có phân phối đuôi nặng hơn và thứ tự tương đối giữa các scheduler có thể thay đổi. (2) *Một topology*: cụm chạy

trên một host Docker duy nhất — độ trễ mạng đối xứng, không lệch đồng hồ, cache hệ thống file chung. (3) *Giả thiết tuyến tính*: cận regret của [10] đòi hỏi $\mathbb{E}[r|x] = x^\top \theta^*$; thời gian biên dịch phi tuyến theo kích thước nguồn, và phép nén log không tuyến tính hoá hoàn toàn — sai đặc tả biên độ ε làm regret tăng cộng $O(\varepsilon \sqrt{T})$ [8]. (4) *LinUCB thường so với SupLinUCB*: bound $\tilde{O}(\sqrt{Td})$ chứng minh cho SupLinUCB; chúng tôi triển khai LinUCB Algorithm 1 đơn giản hơn, không có bound đã chứng minh. (5) *Drift*: không cơ chế nào (sliding window, phát hiện change-point) được cài để xử lý trôi dạt hiệu năng worker. (6) *Không so với hệ sản xuất*: so sánh giới hạn ở các thuật toán chúng tôi cài đặt, không phải bộ lập lịch bên trong Bazel RBE hay Incredibuild.

7.4. Các mối đe dọa đến tính hợp lệ

Nội tại: nhiều run-to-run của Docker trên một host là mối đe dọa chính; thiết kế khối ngẫu nhiên hoá cùng kiểm định ghép cặp được chọn chính để trung hoà nó, và các bảng single-run được dán nhãn rõ là bối cảnh không kết luận. *Ngoại tại*: một workload, một topology; thêm nữa cả bốn scheduler trong đánh giá chính chạy cùng cấu hình phần cứng, nên kết luận không tự động mở rộng sang cụm quy mô khác. *Cấu trúc*: makespan build sạch là chỉ tiêu chính; nếu mục tiêu triển khai là độ trễ đuôi hoặc incremental build, thứ hạng scheduler có thể khác — dữ liệu P99 của chúng tôi (không phân biệt được) nhấn mạnh điều này.

8. Kết luận và hướng phát triển

Bài báo trình bày Hybrid-Grid — hệ thống biên dịch phân tán hoàn chỉnh bằng Go với sáu bộ lập lịch sau một giao diện học hợp nhất — và một nghiên cứu thực nghiệm có kiểm soát nhiều về lập lịch contextual bandit trên cụm không đồng nhất. Kết quả trung thực gồm hai phần. Về *kết quả*: Hybrid-LinUCB (warm-start + phạt tải) vượt có ý nghĩa các bộ học yếu hơn nhưng chỉ ngang bằng heuristic LeastLoaded trên workload biên dịch dừng — thể hoà được chứng minh là bản chất qua ablation warm-bandit, không phải hệ quả khởi động lạnh. Về *phương pháp luận*: quy trình khối ngẫu nhiên hoá với kiểm định ghép cặp đã phát hiện và loại bỏ một kết quả dương tính giả do nhiễu thứ tự chạy — bài học có giá trị độc lập cho cộng đồng đo lường hệ thống.

Hướng phát triển bám sát chẩn đoán “khi nào bộ học thắng”: (i) lập lịch *cache-aware* trên workload incremental rebuild — thêm đặc trưng “worker đã có artifact trong cache” mà coordinator suy ra được từ lịch sử dispatch không cần RPC mới; (ii) thích ứng *drift* bằng phát hiện change-point (CUSUM, Page-Hinkley) với reset cục bộ A_a, b_a ; (iii) phần thưởng dạng Decima $-(t_k - t_{k-1})J_k$ có biện luận từ định luật Little thay cho log-latency thuần kỹ nghệ; (iv) mở rộng véc-tơ ngữ cảnh khi các đường build Flutter/Unity đi vào luồng học. Toàn bộ mã nguồn, log đo thô và script tái lập được công bố cùng hệ thống.

Lời cảm ơn

[Bổ sung thông tin tài trợ, hỗ trợ của đơn vị và giáo viên hướng dẫn tại đây.]

Tuyên bố về sử dụng AI tạo sinh trong quá trình viết

Trong quá trình chuẩn bị công trình này, nhóm tác giả có sử dụng công cụ hỗ trợ AI để cải thiện chất lượng ngôn ngữ và độ rõ ràng của văn bản. Nhóm tác giả đã rà soát, biên tập nội dung khi cần và chịu hoàn toàn trách nhiệm về nội dung của công bố.

Tài liệu tham khảo

- [1] J. K. Lenstra, D. B. Shmoys, É. Tardos, Approximation algorithms for scheduling unrelated parallel machines, *Mathematical Programming* 46 (1990) 259–271. doi:10.1007/BF01585745.
- [2] R. L. Graham, Bounds on multiprocessing timing anomalies, *SIAM Journal on Applied Mathematics* 17 (2) (1969) 416–429.
- [3] M. Mitzenmacher, The power of two choices in randomized load balancing, *IEEE Transactions on Parallel and Distributed Systems* 12 (10) (2001) 1094–1104. doi:10.1109/71.963420.
- [4] K. Ousterhout, P. Wendell, M. Zaharia, I. Stoica, Sparrow: distributed, low latency scheduling, in: *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 69–84. doi:10.1145/2517349.2522716.
- [5] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Transactions on Parallel and Distributed Systems* 13 (3) (2002) 260–274.
- [6] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [7] A. Slivkins, Introduction to multi-armed bandits, *Foundations and Trends in Machine Learning* 12 (1–2) (2019) 1–286. arXiv:1904.07272.
- [8] T. Lattimore, C. Szepesvári, *Bandit Algorithms*, Cambridge University Press, 2020.
- [9] L. Li, W. Chu, J. Langford, R. E. Schapire, A contextual-bandit approach to personalized news article recommendation, in: *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 661–670. doi:10.1145/1772690.1772758.
- [10] W. Chu, L. Li, L. Reyzin, R. E. Schapire, Contextual bandits with linear payoff functions, in: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 208–214.
- [11] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016, pp. 50–56. doi:10.1145/3005745.3005750.

- [12] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, in: Proceedings of ACM SIGCOMM, 2019, pp. 270–288. doi:10.1145/3341302.3342080.
- [13] C. Delimitrou, C. Kozyrakis, Quasar: resource-efficient and QoS-aware cluster management, in: Proceedings of ASPLOS, 2014, pp. 127–144. doi:10.1145/2541940.2541941.
- [14] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, R. Bianchini, Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms, in: Proceedings of SOSP, 2017, pp. 153–167. doi:10.1145/3132747.3132772.
- [15] M. Pool, distcc: a fast, free distributed C/C++ compiler, 2004. <https://distcc.github.io/>.
- [16] P. Auer, N. Cesa-Bianchi, P. Fischer, Finite-time analysis of the multiarmed bandit problem, Machine Learning 47 (2–3) (2002) 235–256.
- [17] J. Sherman, W. J. Morrison, Adjustment of an inverse matrix corresponding to a change in one element of a given matrix, The Annals of Mathematical Statistics 21 (1) (1950) 124–127.
- [18] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [19] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
- [20] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, Journal of the American Statistical Association 32 (200) (1937) 675–701.
- [21] N. Cliff, Dominance statistics: ordinal analyses to answer ordinal questions, Psychological Bulletin 114 (3) (1993) 494–509.